A good **SOP (Standard Operating Procedure)** explains exactly how your project works from beginning to end. You can use the following for your capstone report or presentation.

SOP – ParkPulse Smart Parking Management System

Project Name

ParkPulse – Smart Parking, Smarter Cities

Objective

The objective of ParkPulse is to help drivers find, reserve, and use parking spaces efficiently by providing real-time parking availability through IoT sensors, cloud technology, and a mobile application.

Phase 1 – User Opens the Application

Step 1: Launch the App

The user opens the ParkPulse mobile application.

Step 2: User Login

The user signs in using:

- Email & Password
- Google Login
- Mobile Number (OTP)

After successful login, the user reaches the home screen.

Phase 2 – Detect User Location

Step 3: Location Permission

The app requests permission to access the user's location.

If permission is granted:

- The current location is detected using GPS.
- Nearby parking locations are displayed on Google Maps.

Example:

Current Location:

- User: Andheri

Nearby Parking:

- Mall Parking – 2 km
- Metro Parking – 1.5 km
- Airport Parking – 5 km

Phase 3 – View Parking Availability

Step 4: Real-Time Parking Status

Each parking slot has:

- Ultrasonic Sensor
- ESP32 Controller

The sensor continuously checks whether a vehicle is present.

If a vehicle is detected:

- Slot = Occupied

Otherwise:

- Slot = Available

The ESP32 sends this information to Firebase.

Firebase updates the mobile application instantly.

The user can see:

Parking A

Available Slots: 18

Occupied Slots: 32

Reserved Slots: 5

Phase 4 – Select Parking

Step 5: Choose Parking Area

The user selects a preferred parking location.

The app displays all parking slots.

Example:

Ground Floor

A1 Available

A2 Occupied

A3 Occupied

A4 Available

A5 Reserved

Phase 5 – Reserve Parking

Step 6: Reserve Slot

The user selects an available parking slot.

The system checks whether the slot is still free.

If available:

- Slot status changes to Reserved.
- Reservation timer starts (15 minutes).

Example:

Slot A4

Reserved for

15 Minutes

Phase 6 – QR Code Generation

Step 7: Booking Confirmation

The system generates:

- Booking ID
- QR Code
- Parking Number
- Entry Time

Example:

Booking ID

PP45876

Parking Slot

A4

Status

Reserved

The QR code is stored in the application.

Phase 7 – Navigation

Step 8: Route Guidance

The user clicks "Navigate."

Google Maps opens automatically.

Navigation begins from the user's current location to the selected parking area.

Phase 8 – Entry Verification

Step 9: Parking Entrance

At the entrance:

The QR code is scanned.

The system verifies:

- Booking ID
- User Information
- Reservation Status
- Reservation Time

If valid:

The gate opens automatically.

Otherwise:

Entry is denied.

Phase 9 – Parking Detection

Step 10: Vehicle Parking

The driver parks in Slot A4.

The ultrasonic sensor measures the distance.

Example:

Distance

5 cm

Vehicle Present

Yes

The ESP32 updates Firebase.

Firebase changes:

A4

Reserved

↓

Occupied

The mobile app updates instantly.

Other users now see A4 as occupied.

Phase 10 – Parking Timer

Step 11: Parking Duration

The system starts calculating parking time.

Example:

Entry

10:15 AM

Exit

12:20 PM

Duration

2 Hours 5 Minutes

The timer runs until the vehicle exits.

Phase 11 – Digital Payment

Step 12: Payment Calculation

When the user leaves:

Parking duration is calculated.

Example:

Rate

₹50 per Hour

Time

2 Hours

Total

₹100

Payment options:

- UPI
- Credit Card
- Debit Card
- Net Banking

After payment:

Payment Status = Success

Phase 12 – Exit

Step 13: Exit Verification

The user scans the QR code again.

The system verifies:

- Payment
- Booking ID
- Exit Time

The gate opens automatically.

Phase 13 – Slot Update

Step 14: Vehicle Leaves

The vehicle exits the parking slot.

The ultrasonic sensor detects no vehicle.

The ESP32 sends updated data to Firebase.

Firebase changes:

Slot A4

Occupied

↓

Available

The mobile app immediately displays the slot as available for other users.

Phase 14 – Admin Dashboard

The administrator can monitor:

- Total Parking Slots
- Available Slots
- Occupied Slots
- Reserved Slots
- Total Revenue
- Daily Bookings
- User Details
- Parking History

Example Dashboard:

Total Slots

100

Occupied

68

Available

24

Reserved

8

Revenue Today

₹18,500

Data Flow

Driver
│
▼
Mobile Application
│
▼
GPS detects location
│
▼
Nearby parking displayed
│
▼
User selects parking
│
▼
Reservation created
│
▼
QR Code generated
│
▼
Google Maps navigation
│
▼

Parking Entrance
|
▼
QR Code verification
|
▼
Gate Opens
|
▼
Vehicle parks
|
▼
Ultrasonic Sensor detects vehicle
|
▼
ESP32
|
▼
Firebase Cloud
|
▼
Mobile App updates
|
▼
Parking timer starts
|
▼
Digital payment
|
▼
Exit verification
|
▼
Vehicle leaves
|
▼
Sensor detects empty slot
|
▼
Firebase updates status
|
▼
Slot becomes available again

Hardware Used

- ESP32
- Ultrasonic Sensor (HC-SR04)
- IR Sensor (optional)
- QR Code Scanner
- Wi-Fi Module (built into ESP32)

Software Used

- React.js or Flutter (Mobile/Web App)
- Node.js + Express (Backend API)
- Firebase Realtime Database
- Firebase Authentication
- Google Maps API
- QR Code Generator
- Payment Gateway (UPI integration)

This SOP covers the complete lifecycle of your ParkPulse project, from the moment a user opens the app to the moment the parking slot becomes available again after they leave. It is suitable for a capstone project report, viva, and implementation guide.